

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad del Perú, Decana de América

Facultad de Ingeniería de Sistemas e Informática



CURSO: Lenguajes y Compiladores

DOCENTE: Prudencio Vidal, Javier Antonio

GRUPO: 5

INTEGRANTES:

Arboleda Sanchez, Ericka Sofia	24200047
Badillo Castillo, Edwin Piero	24200150
Ccoicca Fernandez, Adrian	24200154
Huamanchahua Huayanay, Oscar	24200161
Maque Espinoza, Steveen Dennys	24200059
Reynoso Castillo, José Luis	24200169
Zarate Vilchez, Axhell Jesus	24200177

Índice

Índice.....	2
1. Introducción.....	3
2. Diseño del Árbol de Sintaxis Abstracta (Semana 9).....	4
2.1. Definición y estructura de nodos.....	4
2.2. Nodos del AST para el lenguaje Dummy.....	4
2.3. Funciones de construcción de nodos.....	6
2.4. Construcción del AST en Bison.....	7
3. Robustez del Parser (Semana 10).....	8
3.1. Precedencia de operadores.....	8
3.2. Pruebas con anidamiento.....	9
3.3. Impresión y validación del árbol.....	10
4. Conclusiones.....	11

1. Introducción

El presente informe documenta el desarrollo de las fases correspondientes a las semanas 9 y 10 del proyecto del compilador del lenguaje Dummy. Estas semanas corresponden a la segunda parte de la Fase 2, centrada en la construcción del Árbol de Sintaxis Abstracta (AST) a partir del análisis sintáctico ya implementado en semanas anteriores, y en la validación de la robustez del parser frente a expresiones complejas y estructuras anidadas.

El AST es la representación interna del programa fuente que utilizará el compilador en todas las fases posteriores: análisis semántico, verificación de tipos y generación de código. A diferencia del árbol de parsing concreto, el AST omite los detalles sintácticos irrelevantes (como los puntos y coma o los paréntesis) y conserva únicamente la estructura lógica del programa. Las dos semanas cubiertas en este informe corresponden a las siguientes actividades según el cronograma del proyecto:

- **Semana 9:** Diseño del Árbol de Sintaxis Abstracta, definición de nodos, construcción del AST directamente en las reglas de Bison, y validación mediante impresión del árbol en consola.
- **Semana 10:** Validación de la precedencia de operadores, pruebas con estructuras anidadas (loop dentro de cuando, expresiones compuestas), y estabilización del parser con el AST integrado.

2. Diseño del Árbol de Sintaxis Abstracta (Semana 9)

2.1. Definición y estructura de nodos

Un Árbol de Sintaxis Abstracta (AST, por sus siglas en inglés) es una representación jerárquica del código fuente donde cada nodo representa una construcción semántica del lenguaje: una declaración, una asignación, una operación aritmética, un condicional, etc. El AST es abstracto porque no registra todos los símbolos del árbol de parsing concreto, sino solo los que tienen significado semántico.

Para el lenguaje Dummy, cada nodo del AST se representa mediante la siguiente estructura en C, definida directamente en el archivo parser.y:

```
typedef struct NodoAST {
    TipoNodo      tipo;
    char          *str_val; /* nombre, operador o tipo de dato */
    int           int_val;  /* valor entero o booleano          */
    struct NodoAST *izq;    /* hijo izquierdo / condicion */
    struct NodoAST *der;    /* hijo derecho / bloque      */
    struct NodoAST *extra;  /* bloque sino / siguiente en lista*/
} NodoAST;
```

El campo tipo identifica qué clase de nodo es. Los campos str_val e int_val almacenan el valor literal asociado al nodo (por ejemplo, el nombre de una variable o el valor de un entero). Los tres punteros izq, der y extra conectan el nodo con sus hijos, permitiendo representar estructuras binarias (operaciones), ternarias (condicional con sino) y listas enlazadas (secuencia de sentencias).

2.2. Nodos del AST para el lenguaje Dummy

La siguiente tabla describe todos los tipos de nodo definidos para representar los constructos del lenguaje Dummy:

Tipo de Nodo	Descripción	Producción gramatical asociada
NODO_PROGRAMA	Raíz del árbol. Representa el bloque inicio { }	izq → lista de sentencias
NODO_LISTA_SENT	Nodo enlazador de sentencias consecutivas	izq → sentencia actual, extra → siguiente
NODO_DECLARACION	Declaración de variable con o sin inicialización	str_val=tipo, izq=id, der=expresión (puede ser NULL)
NODO_ASIGNACION	Reasignación de variable ya declarada	izq → identificador, der → expresión
NODO_CONDICIONAL	Estructura cuando con rama opcional sino	izq=condición, der=bloque_si, extra=bloque_sino
NODO_CICLO	Estructura loop con condición y cuerpo	izq → condición, der → cuerpo
NODO_IMPRESION	Llamada a imprimir con una expresión	izq → expresión a imprimir
NODO_RETORNO	Instrucción retornar con valor opcional	izq → expresión (puede ser NULL)
NODO_OP_BINARIO	Operación aritmética, relacional o lógica binaria	str_val=operador, izq=operando_izq, der=operando_der
NODO_OP_UNARIO	Operación unaria: negación lógica (!) o menos unario	str_val=operador, izq=operando
NODO_ENTERO	Literal entero	int_val → valor
NODO_DECIMAL	Literal decimal	str_val → valor como cadena

NODO_TEXTO	Literal de cadena de texto	str_val → contenido
NODO_BOOLEANO	Literal booleano (verdadero/falso)	int_val → 1 o 0
NODO_IDENTIFICADOR	Referencia a una variable	str_val → nombre

2.3. Funciones de construcción de nodos

Para cada tipo de nodo se implementa una función de construcción dedicada. Todas siguen el mismo patrón: reservan memoria con malloc, inicializan todos los campos a cero o NULL, y retornan el puntero al nodo creado. Las principales son:

```

NodoAST* nuevo_nodo(TipoNodo tipo)      /* aloca e inicializa */
NodoAST* nodo_entero(int val)
NodoAST* nodo_decimal(char *val)
NodoAST* nodo_texto(char *val)
NodoAST* nodo_booleano(int val)
NodoAST* nodo_id(char *nombre)
NodoAST* nodo_op_binario(char *op, NodoAST *izq, NodoAST *der)
NodoAST* nodo_op_unario(char *op, NodoAST *operando)
NodoAST* nodo_declaracion(char *tipo, char *nombre, NodoAST *expr)
NodoAST* nodo_asignacion(char *nombre, NodoAST *expr)
NodoAST* nodo_condicional(NodoAST *cond, NodoAST *si, NodoAST *sino)
NodoAST* nodo_ciclo(NodoAST *cond, NodoAST *cuerpo)
NodoAST* nodo_impresion(NodoAST *expr)
NodoAST* nodo_retorno(NodoAST *expr)
NodoAST* nodo_lista(NodoAST *sentencia, NodoAST *siguiente)

```

Adicionalmente se implementan dos funciones de soporte que recorren el árbol recursivamente:

- **imprimir_ast(NodoAST *nodo, int nivel):** recorre el árbol en preorden e imprime cada nodo con sangría proporcional a su profundidad, permitiendo verificar visualmente la estructura construida.
- **liberar_ast(NodoAST *nodo):** libera toda la memoria reservada por el árbol de forma recursiva, evitando memory leaks al finalizar el análisis.

2.4. Construcción del AST en Bison

La integración del AST con Bison requirió los siguientes cambios en el archivo parser.y respecto a la versión de semana 8:

- **Extensión del %union:** se agrega el campo nodo de tipo NodoAST* para que las reglas gramaticales puedan retornar nodos del árbol como valor semántico:

```
%union {
    int entero;
    char *str;
    struct NodoAST *nodo;    /* nuevo campo para el AST */
}
```

- **Declaración de tipos con %type:** cada no terminal que construye un nodo declara su tipo de retorno:

```
%type <nodo> sentencias sentencia declaracion asignacion
%type <nodo> condicional ciclo impresion retorno condicion expresion
%type <str> tipo operador_rel
```

- **Reemplazo de printf por construcción de nodos:** cada acción semántica que antes imprimía un mensaje de debug ahora construye y retorna el nodo correspondiente. Por ejemplo, la regla de asignación:

```
asignacion:
    IDENTIFICADOR OP_ASIGNACION expresion
```

```

{
    $$ = nodo_asignacion($1, $3);
};

```

- **Ensamblaje en la regla raíz:** la regla programa crea el nodo raíz, invoca imprimir_ast para mostrar el árbol completo en consola, y libera la memoria al terminar:

```

programa:
    KW_INICIO DELIM_LLAVE_IZQ sentencias DELIM_LLAVE_DER
    {
        NodoAST *raiz = nuevo_nodo(NODO_PROGRAMA);
        raiz->izq = $3;
        imprimir_ast(raiz, 0);
        liberar_ast(raiz);
    };

```

3. Robustez del Parser (Semana 10)

3.1. Precedencia de operadores

La precedencia de operadores estaba declarada en el parser desde la semana 7, pero en la versión anterior las acciones semánticas eran solo printf, por lo que no era posible verificar que el árbol se construyera con la asociatividad correcta. Con el AST implementado, la precedencia se puede validar directamente observando la estructura del árbol generado. La tabla de precedencia vigente, de menor a mayor, es:

Precedencia	Operadores	Asociatividad
1 (menor)	OP_OR ()	%left
2	OP_AND (&&)	%left
3	OP_IGUAL OP_DIFERENTE OP_MAYOR OP_MENOR OP_MAYOR_IGUAL OP_MENOR_IGUAL	%left

4	OP_SUMA OP_RESTA (+ -)	%left
5	OP_MULT OP_DIV OP_MOD (* / %)	%left
6 (mayor)	OP_NOT (!)	%right

Gracias a estas declaraciones, Bison construye el árbol respetando la precedencia matemática estándar. Por ejemplo, la expresión $2 + 3 * 4$ genera un nodo `NODO_OP_BINARIO` con operador `+` cuyo hijo derecho es otro `NODO_OP_BINARIO` con operador `*`, reflejando que la multiplicación se evalúa primero,

3.2. Pruebas con anidamiento

Se realizaron pruebas con programas que combinan estructuras anidadas para verificar que el AST se construya correctamente en todos los casos. Los escenarios probados incluyen:

- Loop dentro de cuando: un condicional cuyo bloque verdadero contiene un ciclo loop. El AST muestra el nodo `NODO_CONDICIONAL` con su hijo `der` (`bloque_si`) apuntando a un `NODO_LISTA_SENT` que contiene un `NODO_CICLO`.
- Expresiones aritméticas compuestas: expresiones como $(a + b) * (c - d)$ donde los paréntesis modifican la estructura del árbol. Al ser consumidos por la regla de paréntesis (que retorna directamente `$2`), el árbol refleja correctamente la precedencia explícita.
- Condicional con rama sino: el nodo `NODO_CONDICIONAL` utiliza el puntero extra para enlazar el bloque sino, que puede a su vez contener cualquier secuencia de sentencias incluyendo otros condicionales.

- Declaración con inicialización vs. declaración simple: ambas formas generan un NODO_DECLARACION; en la declaración simple el puntero der queda en NULL, lo que se verifica en la impresión del árbol.

3.3. Impresión y validación del árbol

La función `imprimir_ast` recorre el árbol en preorden y produce una salida con sangría que representa visualmente la jerarquía del programa analizado. Por ejemplo, para el programa:

```
inicio {
    num x = 5;
    cuando (x > 0) {
        imprimir(x);
    }
}
```

La salida del AST en consola es:

```
[PROGRAMA]
[DECLARACION] tipo=num var=x
[VALOR INICIAL]
[LIT_ENTERO] 5
[CUANDO]
[CONDICION]
[OP_BIN] >
[ID] x
[LIT_ENTERO] 0
[BLOQUE_SI]
[IMPRIMIR]
[ID] x
```

Esta representación permite verificar que el árbol tiene la estructura esperada antes de avanzar a las fases de análisis semántico y generación de código.

4. Conclusiones

El desarrollo de las semanas 9 y 10 completó la Fase 2 del compilador del lenguaje Dummy. La construcción del AST representa un paso cualitativo fundamental respecto a las semanas anteriores: el parser ya no solo verifica que el programa sea sintácticamente correcto, sino que produce una representación interna estructurada que será la base de todas las fases siguientes.

La integración del AST con Bison se realizó mediante tres cambios principales en el archivo parser.y: la extensión del %union para incluir el tipo NodoAST*, la declaración de tipos de retorno con %type para cada no terminal relevante, y el reemplazo de los printf de debug por llamadas a las funciones de construcción de nodos. El resto del archivo, incluyendo los tokens, la precedencia y la estructura de las reglas gramaticales, se mantuvo sin cambios.

La función imprimir_ast permite verificar visualmente la corrección del árbol construido, lo que facilita la detección de errores antes de avanzar a la siguiente fase. La función liberar_ast garantiza que la memoria reservada durante el análisis sea liberada correctamente al terminar.

El compilador se encuentra ahora en condiciones de avanzar hacia la Fase 3, correspondiente al análisis semántico: construcción de la tabla de símbolos, verificación de variables declaradas, y comprobación de tipos. Estas actividades están previstas para las semanas 11 y 12 según el cronograma del proyecto.